

MIAE 215 · Programming for Mechatronics & Systems

Assignment 3 & Week 5 In-Person Lecture Problems — Fully Solved Master Guide

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering
Course Engineering Hub · Academic Year 2025/2026

Executive Summary & Curriculum Scope

This guide works through every program in two new course packs:

- **Assignment 3** (with the official solutions): three problems that make you apply the 7-step *program organization* method from `program_organization.txt`:
 1. **Q1**: draw `example5_statistics2` (maximum of a sequence) in **Flowgorithm** and label each organization step on the flowchart.
 2. **Q2**: repeatedly evaluate an n th-order polynomial until a negative n is entered.
 3. **Q3**: approximate e^x with its **Taylor series**, adding terms until the error drops below a tolerance.
- **Week 5, Lecture 2 (in person)**, `control_statement_examples_part3`:
 1. **Example 6 (vector addition)**: arrays used as mathematical vectors with indices $1 \dots n$.
 2. **Example 6 (array reverse)**: sentinel-terminated array input and the reverse-index formula $A[n-1-i]$.
 3. **Example 7 (file output and plotting)**: writing a CSV table of $\sin t$ with `ofstream` and plotting it in a spreadsheet.

Every program below was compiled with the course's portable MinGW `g++` and run. The console outputs shown are real runs, not hand-written.

Program	Main "Lego" (reusable pattern)	New idea introduced
A3 Q1	<code>for</code> loop + "bigger and better deal" max search	Flowchart ↔ code ↔ 7 steps mapping
A3 Q2	<code>while(1)</code> + <code>break</code> on a sentinel; running power <code>prod *= x</code>	Polynomial evaluation without <code>pow</code>
A3 Q3	<code>while(1)</code> until a tolerance is met; running <code>*=</code> products	Series convergence, <code>double</code> for factorials
W5 Ex 6a	<code>for(i=1;i<=n;i++)</code> over vectors	1-based vector indexing, array size $n + 1$
W5 Ex 6b	Sentinel input with <code>break</code> , then <code>n = i</code>	Reverse mapping <code>A_rev[i] = A[n-1-i]</code>
W5 Ex 7	<code>for</code> over a real-valued parameter <code>t += dt</code>	File output in 5 steps, CSV format

1. The 7-Step Program Organization (Recap)

Assignment 3 asks you to **label each step in your code exactly like** `example5_statistics2` (e.g. `// 3. INPUT`). The steps, from `program_organization.txt`:

#	Step	What you do	Exam tip
1	DECLARE	List every variable the problem statement implies, at the top of <code>main</code>	Leave room to add more later
2	INITIALIZE	Give variables starting values (constants, expressions or inputs)	Uninitialized variables hold garbage
3	INPUT	Keyboard, file, sensor, constant parameters	Identify inputs from the problem wording
4	CONTROL STATEMENTS	Choose the loops and <code>if</code> s: the program's skeleton	Write empty templates first, fill in later
5	EXPRESSIONS	Equations given by the problem, placed inside the skeleton	Reuse "Lego" blocks from earlier examples
6	OUTPUT	Screen, file, graphics, actuators	Identify outputs from the problem wording
7	DEBUG / TEST	Run known cases and compare	Always check your small hand example

The steps do not have to appear strictly in order in the final code. Input often happens *inside* a control statement (step 3 inside step 4), and an expression may sit inside an `if`. That is why the solutions use combined labels such as `// 2. INITIALIZE, 3. INPUT` or `// 4. and 5.`. **Part marks on tests come from showing this structure**, even when the final answer has a bug.

For the full treatment of the method, see the comprehensive topic guide [Part 5 - Program Architecture, Output Tracing & Algorithmic Patterns](#).

2. Assignment 3, Q1: Flowgorithm and the Organization Steps

Problem statement

Use Flowgorithm to write the program for `example5_statistics2`. Then print out the flow chart and circle / label each step in the program (e.g. `3. INPUT`). From this result you can see that the program organization steps of the notes are analogous to the Flowgorithm components. The implication is that these steps cover a fairly general class of programs across many different languages.

`example5_statistics2` is the Week 5 Lecture 1 problem: *input $n \leq 100$ doubles from the keyboard and print the largest one.*

The official labelled flowchart

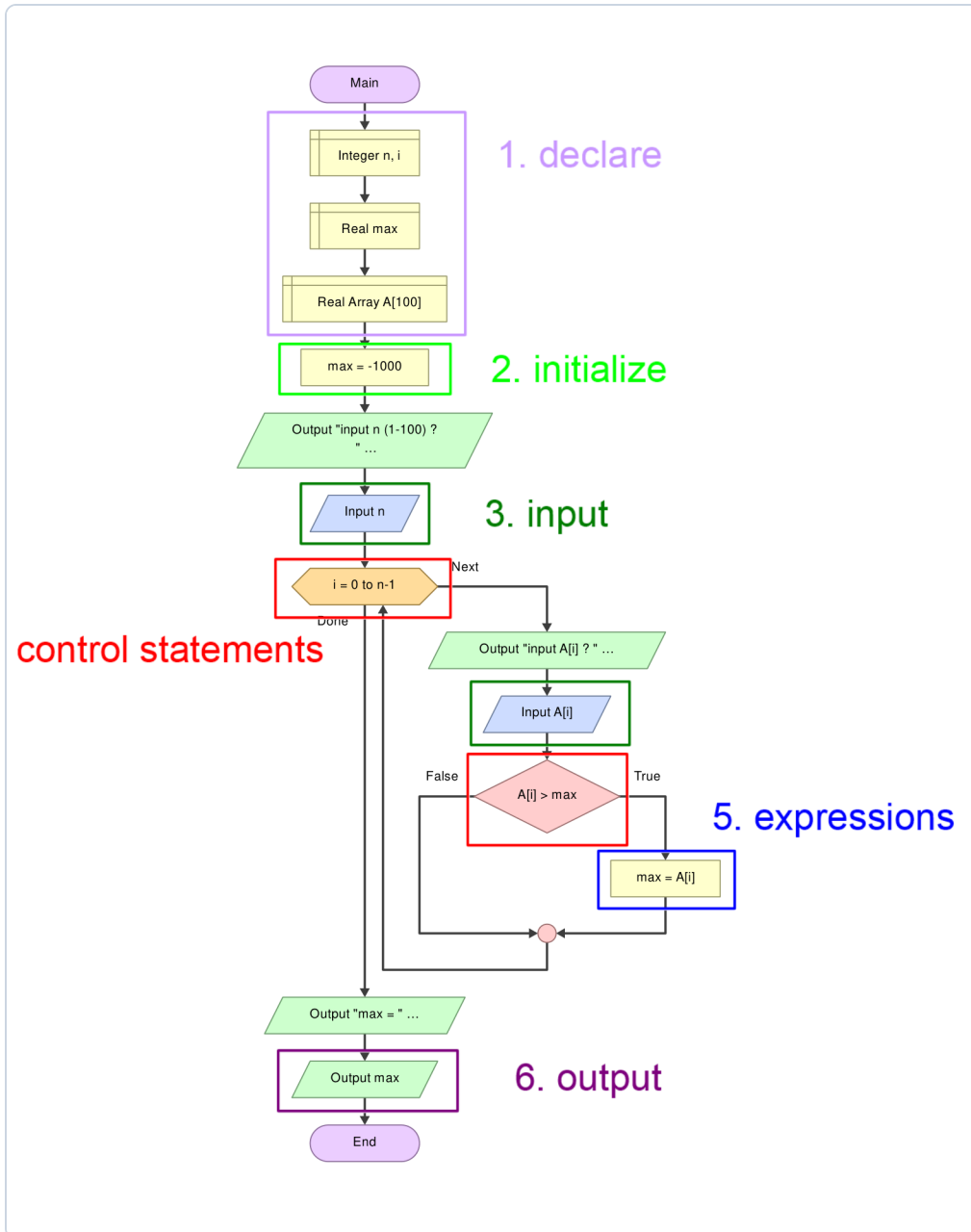


Figure 1: Official Assignment 3 Q1 solution ([q1/q1.pdf](#) , made from [q1.fprg](#)). The red label on the left reads "4. control statements"; its "4." is cut off at the page edge in the original file.

Reading Figure 1 shape by shape.

Flowgorithm shape	Colour in Figure 1	Contents	Organization step
Declare (yellow box with double edge)	Purple frame	Integer <code>n, i</code> · Real <code>max</code> · Real Array <code>A[100]</code>	1. DECLARE
Assign (yellow rectangle)	Green frame	<code>max = -1000</code>	2. INITIALIZE
Input (blue parallelogram)	Dark-green frames	Input <code>n</code> , then Input <code>A[i]</code> inside the loop	3. INPUT
For (orange hexagon) and If (pink diamond)	Red frames	<code>i = 0 to n-1, A[i] > max</code>	4. CONTROL STATEMENTS
Assign inside the <code>True</code> branch	Blue frame	<code>max = A[i]</code>	5. EXPRESSIONS
Output (green parallelogram)	Purple frame	Output <code>max</code>	6. OUTPUT

Some details worth noticing:

- The **"Next" arrow** leaves the hexagon to the right and runs the loop body; the **"Done" arrow** goes down to the output once `i` passes `n-1`. Flowgorithm's `i = 0 to n-1` is **inclusive** of `n-1`, which is exactly C++'s `for(i=0;i<n;i++)`.
- The two branches of the diamond meet again at the small **pink connector circle**, and the line from it returns to the top of the loop. That return line is the `}` that closes the `for` body.
- Input `A[i]` happens **inside** the loop, so step 3 sits inside step 4, just as in the code.
- The prompt outputs (Output "input n (1-100) ? ") are not labelled. They are user-interface text, not the program's answer.
- Step 7 (DEBUG / TEST) has no shape. It is something *you* do by running the flowchart, which is why it is not on the diagram.

The point of the exercise. Flowgorithm uses one shape per kind of action, and those kinds line up one-to-one with the organization steps. The steps therefore describe *any* imperative program, in C++, Python, MATLAB or a flowchart. That is why they are a good high-level plan when you face a blank page on an exam.

The C++ program (official solution, [Q1/example5_statistics2/program.cpp](#))

```
int main()
{
    // 1. DECLARE
    int n, i;
    double A[100], max;

    // 2. INITIALIZE, 3. INPUT
    cout << "\ninput n (1-100) ? ";
    cin >> n;

    // note pad -- record the current maximum value
    max = -1000; // low value will be replaced immediately
    // max = A[0]; // alternative -- start with the first element

    for(i=0;i<n;i++) {
        cout << "\ninput A[i] ? ";
        cin >> A[i]; // 3. INPUT

        // 4. and 5.
        if( A[i] > max ) max = A[i];
    }

    // 6. OUTPUT
    cout << "\nmax = " << max;

    // 7. DEBUG / TEST
    // at least check your simple handwritten example

    cout << "\ndone.\n";
    getchar();
    return 0;
}
```

Whiteboard trace with the teacher's hand example $A = \{1.2, 5.3, 1.7\}$

Iteration	i	A[i] read	Test A[i] > max	max after
start	–	–	–	–1000
1	0	1.2	$1.2 > -1000 \rightarrow \text{true}$	1.2
2	1	5.3	$5.3 > 1.2 \rightarrow \text{true}$	5.3
3	2	1.7	$1.7 > 5.3 \rightarrow \text{false}$	5.3

Real run (input 3, 1.2, 5.3, 1.7):

```
input n (1-100) ?
input A[i] ?
input A[i] ?
input A[i] ?
max = 5.3
done.
```

Initialization trap. `max = -1000` fails if every input is below -1000 : for $A = \{-5000, -2000\}$ the program prints -1000 , which is not even in the data. The Week 5 lecture version uses `max_A = -1e10` (much safer), and the fully robust version starts from the first element: `max = A[0]`; then loops from `i = 1`. That requires `A[0]` to be read *before* the search, so it needs a separate input loop first.

3. Assignment 3, Q2: Repeated Polynomial Evaluation

Problem statement

Write a program that repeatedly evaluates an n -th order polynomial $p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$, where $n \leq 10$. The program inputs n , a_i and x from the keyboard and prints the corresponding value of p . It continues to input new values of n , a_i , x and evaluates p until a negative value for n is input, at which point the program stops.

Planning with the 7 steps

Step	Decision	Reasoning
1. DECLARE	<code>int n, i; double p, x, prod; double a[11];</code>	$n \leq 10$ means coefficients $a_0 \dots a_{10}$: 11 elements
2. INITIALIZE	Nothing at the top	Every variable is set by input or reset inside the loop
3. INPUT	<code>n, x</code> , then <code>a[0] ... a[n]</code>	Read inside the repeating loop
4. CONTROL	<code>while(1)</code> with <code>if(n < 0) break;</code> and a <code>for</code> for the sum	Unknown number of repetitions means an infinite loop with an exit
5. EXPRESSIONS	<code>p = a[0]; prod = 1.0;</code> then <code>prod *= x; p += a[i]*prod;</code>	Running power avoids <code>pow</code>
6. OUTPUT	<code>cout << "\np(x) = " << p;</code>	Once per polynomial
7. TEST	$n = 2, a = [1, 2, 3], x = -1$ gives $p = 2$	Hand-checkable case

Official solution ([Q2/CodeBlocks/program.cpp](#) , core)

```
int main()
{
    // 1. DECLARE
    int n, i;
    double p, x, prod;
    double a[11]; // need 11 since n can go from 0 to 10 -- 11 elements

    // 2. INITIALIZE
    // - nothing required since inputs take care of this later

    // 4. CONTROL STATEMENTS
    while(1) { // use infinite loop to repeatedly input x, ai, n

        // 3. INPUT
        cout << "\ninput n (1-10) ? ";
        cin >> n;

        // put the program-ending if as close as possible to where n is set
        if(n < 0) break;

        // not required, but protects a[i] from going out of range
        if(n > 10) {
            cout << "\nerror n out of range , n = " << n;
            n = 10;
        }

        cout << "\ninput x ? ";
        cin >> x;

        cout << "\ninput a[i]? \n";
        for(i=0;i<=n;i++) cin >> a[i]; // note i goes up to i=n

        // 5. EXPRESSIONS
        p = a[0];
        prod = 1.0; // alternative approach to pow
        for(i=1;i<=n;i++) {
            //          p += a[i]*pow(x,i);
            prod *= x; // calculate x^i
            p += a[i]*prod;
        }

        // 6. OUTPUT
        cout << "\np(x) = " << p;
    }
}
```

```

    }

    // 7. DEBUG / TEST
    // n = 2, a = [1 2 3], x = -1 -> p = 1 + 2*(-1) + 3*(-1)^2 = 2

    cout << "\ndone.\n";
    getchar();
    return 0;
}

```

Why each design choice matters

1. **a[11], not a[10]**. A degree- n polynomial has $n + 1$ coefficients. With $n = 10$ you need indices $0 \dots 10$. This off-by-one is the most common mistake on this question.
2. **i <= n in both loops**. The input loop must read a_n . The sum loop must include the $a_n x^n$ term. Writing **i < n** silently drops the leading term.
3. **The break sits right after cin >> n**. If the test came later, the program would first ask for **x** and coefficients for a polynomial of negative degree. Putting the exit test as close as possible to where its variable is set is a habit worth keeping.
4. **Running power prod *= x**. At iteration i , **prod** holds x^i , built by one multiplication from x^{i-1} . This is cheaper than calling **pow(x,i)** every time (which recomputes the power from scratch) and avoids any doubt about **pow** with a negative base. It is the same "use ***=** in a loop" Lego that Q3's hint asks for.
5. **Range check on n**. The problem did not require it, but without it **n = 15** would write past the end of **a[11]** (undefined behaviour, often a crash). Clamping to 10 keeps the program alive.

Whiteboard trace: $n = 2, a = [1, 2, 3], x = -1$

Stage	i	prod	Term a[i]*prod	p
Before loop	–	1	–	$a_0 = 1$
Iteration 1	1	–1	$2 \times (-1) = -2$	–1
Iteration 2	2	1	$3 \times 1 = 3$	2

Check: $p(-1) = 1 + 2(-1) + 3(-1)^2 = 1 - 2 + 3 = 2 \checkmark$.

Real run (inputs: 2, -1, 1 2 3, then 3, 2, 1 0 -2 1, then -1):

```
input n (1-10) ?
input x ?
input a[i] ?
```

```
p(x) = 2
input n (1-10) ?
input x ?
input a[i] ?
```

```
p(x) = 1
input n (1-10) ?
done.
```

Second polynomial check: $p(2) = 1 + 0(2) - 2(2^2) + 1(2^3) = 1 - 8 + 8 = 1 \checkmark$. The third entry `-1` ends the loop.

Beyond the course (optional): Horner's method. Rewriting $p(x) = a_0 + x(a_1 + x(a_2 + \dots + x a_n))$ gives the shortest loop of all: `p = a[n]; for(i=n-1;i>=0;i--) p = p*x + a[i];`. It uses one multiply and one add per coefficient and is the standard way numerical libraries evaluate polynomials.

Small inconsistency in the prompt. The prompt says `(1-10)`, but $n = 0$ (a constant polynomial $p = a_0$) is valid and handled correctly: the sum loop simply runs zero times.

4. Assignment 3, Q3: Taylor Series for e^x

Problem statement

Calculate e^x with the Taylor series $\text{exp_approx} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$, where `x` and `error` are input from the keyboard and enough terms are used that $\text{abs}(\text{exp_approx} - \text{exp}(x)) < \text{error}$. *Hint:* use `*=` in a loop to calculate x^i and $i!$ and print them to verify your loop before computing the series.

The mathematics

$$e^x = \sum_{i=0}^{\infty} \frac{x^i}{i!}, \quad \text{term}_i = \frac{x^i}{i!}$$

Both the numerator and denominator can be built **incrementally**: going from term $i - 1$ to term i needs one multiplication each.

$$x^i = x^{i-1} \cdot x \qquad i! = (i-1)! \cdot i$$

That is exactly the `*=` hint: `x_power *= x;` and `i_factorial *= i;`.

Official solution ([Q3/CodeBlocks/program.cpp](#) , core)

```
int main()
{
    // 1. DECLARE
    int i;
    double exp_approx, error, x, x_power, i_factorial;

    // 2. INITIALIZE
    i = 0;
    x_power = 1.0;    // x^0
    i_factorial = 1.0; // 0! -- use a double since it could be big
    exp_approx = 1.0; // the i = 0 term

    // 3. INPUT
    cout << "\ninput x ? ";
    cin >> x;
    cout << "\ninput error (> 0) ? ";
    cin >> error;

    // 4. CONTROL STATEMENTS
    // unknown number of terms -> infinite loop, break when done
    while(1) {

        // 5. EXPRESSIONS
        i++;
        i_factorial *= i;
        x_power *= x;
        exp_approx += x_power / i_factorial;

        // 6. OUTPUT, 7. DEBUG / TEST
        cout << "\n\nexp_approx = " << exp_approx;
        cout << "\nexp(x) = " << exp(x);
        cout << "\nabs( exp_approx - exp(x) ) = " << abs( exp_approx - exp(x) );
        cout << "\ni = " << i;
        cout << "\nx_power = " << x_power;
        cout << "\ni_factorial = " << i_factorial;

        // stop when the approximation error is below the desired error
        if( abs( exp_approx - exp(x) ) < error ) break;
    }

    cout << "\ndone.\n";
    getchar();
}
```

```
    return 0;
}
```

Why each design choice matters

1. **Initialization encodes the $i = 0$ term.** Starting with `exp_approx = 1`, `x_power = 1` (x^0) and `i_factorial = 1` ($0!$) means the loop only has to add terms $i = 1, 2, \dots$
2. **Order inside the loop.** `i++` comes first, then both products are updated, and only then is the term added. Swapping the order (e.g. adding before updating) shifts every term by one index.
3. **`i_factorial` is a `double`.** Factorials explode: $12! = 479\,001\,600$ still fits in a 32-bit `int` (max $2\,147\,483\,647$), but $13! = 6\,227\,020\,800$ does not. An `int` factorial would overflow and wrap to garbage for x values that need 13+ terms. A `double` reaches about $170!$ before overflowing.
4. **`while(1)` + `break`.** The number of terms is unknown in advance, so a `for` loop is the wrong Lego. The teacher prefers testing at the end of the body so that at least one term is always added and the exit timing is explicit. `while( abs(exp_approx - exp(x)) >= error )` would also work.
5. **`abs` on a `double`.** With `<cmath>` included, `abs(double)` returns a `double`. In old C code `abs` is integer-only, and `fabs` is the safe choice if in doubt.
6. **Printing `x_power` and `i_factorial`** is step 7 built into the loop: if x^i or $i!$ is wrong you see it on the first lines, before trusting the sum.

Real run: $x = 1.1$, `error` = 0.1 (the teacher's suggested test)

```
exp_approx = 2.1
exp(x) = 3.00417
abs( exp_approx - exp(x) ) = 0.904166
i = 1
x_power = 1.1
i_factorial = 1

exp_approx = 2.705
exp(x) = 3.00417
abs( exp_approx - exp(x) ) = 0.299166
i = 2
x_power = 1.21
i_factorial = 2

exp_approx = 2.92683
exp(x) = 3.00417
abs( exp_approx - exp(x) ) = 0.0773327
i = 3
x_power = 1.331
i_factorial = 6
done.
```

i	$x_power = 1.1^i$	i_factorial	Term added	exp_approx	Error	Stop?
0 (init)	1	1	–	1	2.004	–
1	1.1	1	1.1	2.1	0.904	no
2	1.21	2	0.605	2.705	0.299	no
3	1.331	6	0.22183	2.92683	0.0773	yes (< 0.1)

Engineering remark. Comparing against the library `exp(x)` is fine for an exercise, but in real code you would not know the exact answer (that is why you are approximating it). The practical stopping rule is "stop when the next term is smaller than the tolerance": `if( abs(x_power / i_factorial) < error ) break;` . For e^x with $x > 0$ the remaining tail is a little larger than the last term, so this is a slightly optimistic but standard criterion.

5. Week 5 Lecture 2, Example 6: Vector Addition with 1-Based Indexing

Problem statement

Input two vectors v_1 and v_2 with 5 components each from the keyboard, compute $v_3 = v_1 + v_2$ and print v_3 . The vector index i should vary from $i = 1$ to $i = 5$, following standard vector notation ($v_1[1]$ to $v_1[5]$).

Key idea: arrays vs. vectors

	C++ array convention	Maths vector convention (this example)
Index range	$0 \dots n - 1$	$1 \dots n$
Declaration for $n = 5$	<code>double v[5];</code>	<code>double v[6];</code> (size $n + 1$, <code>v[0]</code> unused)
Loop	<code>for(i=0;i<n;i++)</code>	<code>for(i=1;i<=n;i++)</code>

Declaring one extra element and ignoring `v[0]` lets the code read exactly like the formula $v_3[i] = v_1[i] + v_2[i]$, $i = 1, \dots, 5$. This matters when you translate engineering formulas written with 1-based subscripts (matrices, finite-element nodes) and want to avoid off-by-one translation errors.

Code ([example6_vector_addition/program.cpp](#) , core)

```
// 1. DECLARE
double v1[6], v2[6], v3[5+1]; // size + 1 for vectors
int i, n = 5;

// 3. INPUT
for(i=1;i<=n;i++) { cout << "\ninput v1[i] ? "; cin >> v1[i]; }
for(i=1;i<=n;i++) { cout << "\ninput v2[i] ? "; cin >> v2[i]; }

// 4. CONTROL STATEMENTS
for(i=1;i<=n;i++) {
    v3[i] = v1[i] + v2[i]; // 5. EXPRESSIONS
}

// 6. OUTPUTS
cout << "\nv3 =\n";
for(i=1;i<=n;i++) cout << v3[i] << "\n";
```

Real run with $v_1 = (1, 2, 3, 4, 5)$ and $v_2 = (10, 20, 30, 40, 50)$:

```
v3 =
11
22
33
44
55
```

Trap. Mixing the conventions is fatal: `double v[5];` with `for(i=1;i<=5;i++)` writes `v[5]` , one past the end of the array. The compiler does not catch it, and the program may appear to work and then corrupt a neighbouring variable. Pick one convention per array and size the array to match.

6. Week 5 Lecture 2, Example 6: Sentinel Input and Array Reversal

Problem statement

(a) Input an n -element array A ($n \leq 10$) one component at a time until a value greater than 10^9 is entered; record n and print A . (b) Compute and print an array A_{rev} containing the elements of A in reverse order. Example: input 1, 2.1, 3.14, $1e10$ gives $n = 3$, $A = \{1, 2.1, 3.14\}$, $A_{\text{rev}} = \{3.14, 2.1, 1\}$.

Code (`example6_array_reverse/program.cpp` , `core`)

```
// 1. DECLARE
double A[10], A_rev[10];
int i, n, nmax = 10;

// 3. INPUT
for(i=0;i<nmax;i++) {
    cout << "\nA[i] = ? ";
    cin >> A[i];
    if( A[i] > 1.0e9 ) break;
}
n = i;

// 6. OUTPUT
cout << "\nA =\n";
for(i=0;i<n;i++) cout << A[i] << "\n";

// 4. CONTROL STATEMENTS
// n = 3:  A_rev[0] = A[2] = A[n-1],  A_rev[1] = A[1],  A_rev[2] = A[0]
for(i=0;i<n;i++) {
    A_rev[i] = A[n-1-i]; // 5. EXPRESSIONS
}

// 6. OUTPUT
cout << "\nA_rev =\n";
for(i=0;i<n;i++) cout << A_rev[i] << "\n";
```

Part (a): why `n = i` gives the count

When the sentinel `1e10` is read into `A[3]`, the `break` fires **before** `i++` runs, so `i` is still 3: the index of the sentinel, which equals the number of real values before it. The sentinel itself sits in `A[3]` but is never used, because every later loop runs only to `i < n`.

If the user enters 10 values without a sentinel, the loop ends normally with `i = nmax = 10`, so `n = 10` and the array is full. This is the "check for errors with n" part of the statement: the `for` limit `nmax` is what stops `A` from overflowing. A stricter version would print an error if `n == nmax` and the last value was not a sentinel.

Part (b): deriving the index formula

The teacher's method is to solve a small case by hand, then generalise:

i	$A_rev[i]$	Written with $n = 3$	General form
0	$A[2]$	$A[3-1-0]$	$A[n-1-i]$
1	$A[1]$	$A[3-1-1]$	$A[n-1-i]$
2	$A[0]$	$A[3-1-2]$	$A[n-1-i]$

The two indices always add up to $n - 1$: $i + (n - 1 - i) = n - 1$. Remembering "the pair sums to $n - 1$ " is the fastest way to rebuild the formula under exam pressure.

Real run (input 1, 2.1, 3.14, 1e10):

```
A =
1
2.1
3.14

A_rev =
3.14
2.1
1
```

Extension: reverse in place (no second array). Swap the pairs from both ends and stop at the middle:
`for(i=0;i<n/2;i++) { temp = A[i]; A[i] = A[n-1-i]; A[n-1-i] = temp; }` Running the loop to $i < n$ instead of $i < n/2$ swaps every pair twice and puts the array back in its original order, a classic trace question.

7. Week 5 Lecture 2, Example 7: File Output and Plotting

Problem statement

Calculate and output to a file a table of $\sin(t)$ for $t = 0$ to 2π in increments of 0.01, with a comma between the columns so a spreadsheet can open and plot it (a .csv, comma-separated value, file).

The 5 file-output steps (memorise these)

Step	Code	Purpose
0	<code>#include <fstream></code>	Library that defines file streams
1	<code>ofstream fout;</code>	Declare an output-file stream (like <code>cout</code> , but for a file). Any name works
2	<code>fout.open("output.csv");</code>	Create the file, or overwrite it if it exists
3	<code>if(!fout) { cout << "\nfile open error"; exit(1); }</code>	Stop if the file could not be opened
4	<code>fout << t << " , " << sin(t) << "\n";</code>	Use exactly like <code>cout</code>
5	<code>fout.close();</code>	Flush buffered data to disk and release the file

Code (`example7_file_output_and_plotting/program.cpp` , core)

```
#include <fstream> // needed for file output (step #0)

int main()
{
    double t, dt = 0.01, PI = 3.14159;

    ofstream fout;           // step #1: declare
    fout.open("output.csv");  // step #2: open

    if( !fout ) {             // step #3: check
        cout << "\nfile open error";
        exit(1);
    }

    fout << "t , sin(t)\n";    // step #4: use like cout
    for(t=0;t<2*PI;t+=dt) {
        fout << t << " , " << sin(t) << "\n";
    }

    fout.close();             // step #5: close

    cout << "\ndone.\n";
    getchar();
    return 0;
}
```

What actually lands in the file

Running the program produced an `output.csv` of **630 lines**: one header plus 629 data rows.

```
t , sin(t)
0 , 0
0.01 , 0.00999983
0.02 , 0.0199987
...
6.27 , -0.0131849
6.28 , -0.0031853
```

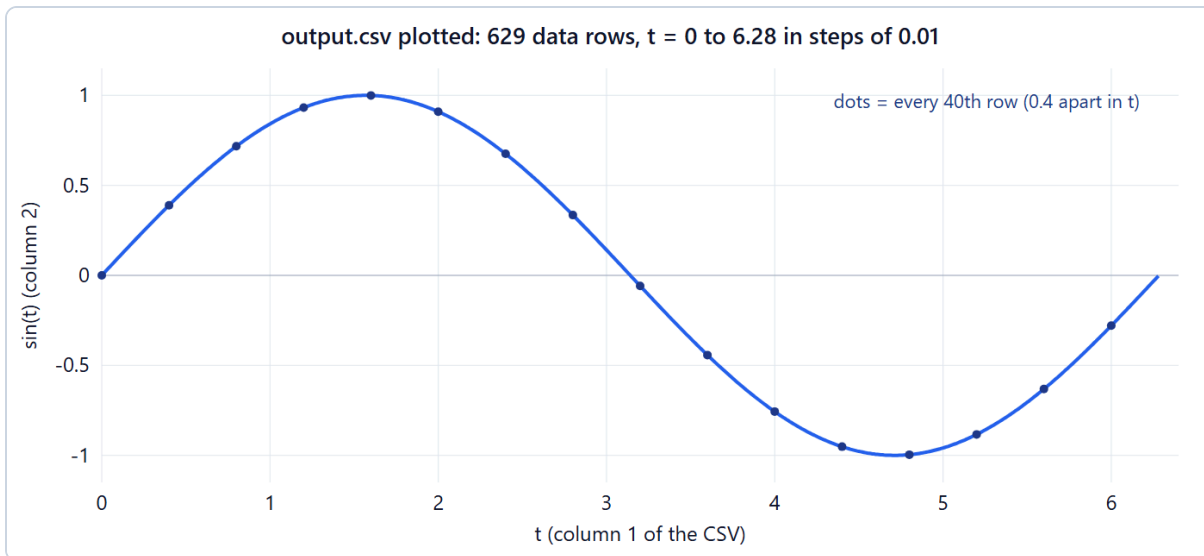


Figure 2: The 629 data rows of `output.csv` from a real run of Example 7, plotted column 1 (t) against column 2 ($\sin t$). This is what the spreadsheet chart in the lecture should look like.

Reading Figure 2. One full period of the sine wave appears: zero at $t = 0$, peak $+1$ near $t = \pi/2 \approx 1.57$, back to zero near $\pi \approx 3.14$, trough -1 near $3\pi/2 \approx 4.71$. The curve ends at $t = 6.28$ with $\sin t \approx -0.003$, just short of zero. It stops before 2π because the loop condition is `t < 2*PI` and $2 \times 3.14159 = 6.28318$, so the last value written is $t = 6.28$. The dots mark every 40th row to show that the "smooth" curve is really a dense table of discrete samples 0.01 apart, which is all a CSV file can hold.

Points examiners like to ask about

1. **Where is the file?** `open` uses a *relative* name, so the file is created in the program's **working directory**. In Code::Blocks or Visual Studio that is usually the project folder, not the folder holding the `.exe`.
2. **Why can `open` fail?** For output, the usual cause is that the file is **already open in another program** (e.g. Excel locks it). A missing file is *not* an error: `open` creates it.
3. **Why `exit(1)`?** There is no point continuing if the results cannot be saved. A nonzero code signals failure to the operating system. `exit` needs `<cstdlib>`.
4. **Why `close`?** Output is buffered in memory. If the program crashes before the buffer is flushed, the end of the data can be lost. `close` also lets you reuse `fout` for another file.

5. **Accumulating `t += dt` in floating point.** 0.01 has no exact binary representation, so after hundreds of additions `t` drifts slightly from the exact multiple of 0.01. For a plot this is harmless. When an exact count matters, loop over an integer `k` and compute `t = k*dt`.
6. **`PI = 3.14159` is an approximation.** For more digits use `4.0*atan(1.0)` or `M_PI` (where the compiler provides it).
7. **The `" , "` separator.** CSV only needs the comma. The spaces are tolerated by spreadsheets, but a strict CSV reader may keep them as part of the value. `fout << t << " , " << sin(t)` is the cleanest form.

8. The Updated C++ "Lego Box"

Lego	Template	Seen in
Maximum search	<code>max = A[0]; for(i=1;i<n;i++) if(A[i] > max) max = A[i];</code>	A3 Q1
Repeat until sentinel	<code>while(1) { cin >> n; if(n < 0) break; ... }</code>	A3 Q2
Running power / factorial	<code>prod = 1; for(...) { prod *= x; } · f = 1; f *= i;</code>	A3 Q2, Q3
Iterate to a tolerance	<code>while(1) { ...update...; if(err < tol) break; }</code>	A3 Q3
1-based vector loop	<code>double v[n+1]; for(i=1;i<=n;i++)</code>	W5 Ex 6a
Sentinel array input	<code>for(i=0;i<nmax;i++){ cin >> A[i]; if(A[i] > big) break; } n = i;</code>	W5 Ex 6b
Reverse mapping	<code>B[i] = A[n-1-i];</code> (indices sum to $n - 1$)	W5 Ex 6b
Write a table to a file	<code>ofstream fout; fout.open("f.csv"); if(!fout) exit(1); fout << ...; fout.close();</code>	W5 Ex 7

9. Common Exam Pitfalls

Pitfall	Consequence	Fix
<code>double a[10]</code> for a degree-10 polynomial	<code>a[10]</code> is out of bounds	Size = $n_{max} + 1$
<code>for(i=1;i<n;i++)</code> in the polynomial sum	Leading term $a_n x^n$ dropped	<code>i <= n</code>
<code>int factorial</code>	Overflow from 13! on	<code>double i_factorial</code>
Updating <code>x_power</code> <i>after</i> adding the term	Every term uses the wrong power	Update products first, then add
Sentinel test placed after other inputs	Program asks for useless inputs before stopping	Test right after reading the sentinel variable
<code>max = 0</code> or <code>max = -1000</code> start value	Wrong answer for all-negative data	Start from <code>A[0]</code> (or a huge negative like <code>-1e10</code>)
<code>double v[5]</code> with <code>i = 1 ... 5</code>	Writes past the array end	Declare <code>v[6]</code> for 1-based use
Forgetting <code>#include <fstream></code>	<code>ofstream</code> undeclared: compile error	Include it (step 0)
No <code>fout.close()</code>	Data can be lost on a crash; file stays locked	Always close
Opening <code>output.csv</code> in Excel and re-running	<code>open</code> fails, "file open error"	Close the spreadsheet first

10. Verification & Compilation Check

All seven programs were compiled with the portable MinGW toolchain shipped with the course's Code::Blocks (`g++ -00`) and run with redirected keyboard input:

Program	Test input	Output	Hand check
A3 Q1	<code>3 · 1.2 5.3 1.7</code>	<code>max = 5.3</code>	✓
A3 Q2	<code>2 · -1 · 1 2 3 · 3 · 2 · 1 0 -2 1 · -1</code>	<code>p(x) = 2</code> , <code>p(x) = 1</code> , then stops	✓
A3 Q3	<code>1.1 · 0.1</code>	stops at <code>i = 3</code> , <code>exp_approx = 2.92683</code>	✓
W5 vector addition	<code>1...5 · 10...50</code>	<code>11 22 33 44 55</code>	✓
W5 array reverse	<code>1 2.1 3.14 1e10</code>	<code>A_rev = 3.14 2.1 1</code>	✓
W5 file output	–	<code>output.csv</code> : 630 lines, last row <code>6.28</code> , <code>-0.0031853</code>	✓

The source files are in `code_solutions/assignment3/` and `code_solutions/week5_lecture2/` next to this guide. To build one from a terminal:

```
g++ -o q2 q2_repeated_polynomial_evaluation.cpp  
q2 < test_input.txt
```